

Academic integrity declaration

By submitting work for assessment, I hereby declare that I understand the University’s policy on academic integrity and the rules of the assignment as set by the subject coordinator regarding the use of artificial intelligence software, including editing and translation tools.

I declare that the submission is original and solely my work, and that I have not been assisted by another person (collusion) apart from where the submitted work is for a designated collaborative task, in which case the individual contributions are indicated. All sources and tools used have been appropriately cited or acknowledged, and any copied or modified code has been declared using the methods indicated by my subject coordinator.

Assignment 1b - Concurrency in FSP

Due: Thu Apr 23, 2026 23:59

8/12.5 Points

Attempt 1



Review Feedback
SUBMITTED: Apr 23, 2026 14:25

Attempt 1 Score:
8/12.5

Add Comment

Anonymous Marking: **no**

Unlimited Attempts Allowed

Details

Weight: 12.5%

Due: 11:59pm, Monday 20 April 2026

Submission: Submit your assignment by uploading a file on this screen and following the prompts.

Questions: Please use the [Ed Discussion Forum \(https://canvas.lms.unimelb.edu.au/courses/237797/external_tools/3724?display=borderless\)](https://canvas.lms.unimelb.edu.au/courses/237797/external_tools/3724?display=borderless) for questions about the assignment - note: please do not post code there!

Extensions: Please see the [FEIT Extensions and Special consideration \(https://canvas.lms.unimelb.edu.au/courses/237797/pages/feit-extensions-and-special-consideration\)](https://canvas.lms.unimelb.edu.au/courses/237797/pages/feit-extensions-and-special-consideration) page.

Academic integrity: Please see the [Academic Integrity \(https://canvas.lms.unimelb.edu.au/courses/237797/pages/academic-integrity\)](https://canvas.lms.unimelb.edu.au/courses/237797/pages/academic-integrity) page.

Introduction

There are two related parts to Assignment 1. This first part of the assignment (1a) dealt with programming concurrency in Java using threads and monitors, and was worth 12.5% of your final mark. The second part of the assignment (1b, the current part) deals with modelling concurrency in FSP and is also worth 12.5% of your final mark. You should complete both parts of Assignment 1 individually.

In the first part of Assignment 1 (which you should complete before attempting this part) you designed and implemented (in Java) a concurrent simulation of a proposed Semaphoria Airport. Now your task is to model that system using FSP and check your model using LTSA. You should refer back to [Assignment 1a](#)

(<https://canvas.lms.unimelb.edu.au/courses/237797/assignments/628788>) for a description of the scenario.

Assignment Tasks

Task 1: Model

The first task is to model the Airport system from Assignment 1a using FSP. Your model should contain comments that explain the design and its components

You may find it helpful to base your model on your Java implementation from Assignment 1a. Alternatively, I will release a sample solution for Assignment 1a after all submissions have been received (noting extensions) on Friday 3 April 2026 that you may use as a starting point. **(Now available from the [Assignment 1a](https://canvas.lms.unimelb.edu.au/courses/237797/assignments/628788) <https://canvas.lms.unimelb.edu.au/courses/237797/assignments/628788> page.)**

Note that there is no scaffold code for Assignment 1b, as there was for 1a, but many of the lecture examples will provide a useful guide to help get you started.

Task 2: Check

Specify what you believe are the relevant safety and liveness properties for your FSP model. Use LTSA to confirm that your model from Task 1 satisfies these properties.

Task 3. Modify

Modify your model from Task 1 to create a variant that *violates* one of your safety or liveness properties from Task 2. Your modification should be a small, clearly identified change that reflects a plausible system design or modelling error.

Be sure to create a copy of the file containing your original FSP model. You are required to submit both the original model from Task 1, and the modified model from Task 3 as separate files.

Task 4: Reflect

Your final task is to write a reflection (strictly between 400 and 500 words) that addresses the following:

- Describe a key model design decision and how it is expressed in your processes and composition.
- Describe an incorrect or unexpected behaviour that you encountered while designing your model and how you resolved it.
- Compare one aspect of your FSP model with your Java implementation. What is easier or harder to express in each, and why?
- Describe the change you made to your model in Task 3, identify the property that is violated, and explain how the violation arises.

In your reflection, you should make explicit reference to your FSP model.

Advice

To tackle the assignment, first work through (and understand) the examples from lectures, and do the workshop exercises. FSP is not difficult—it is simpler than most programming languages, and much simpler than languages like Java. However, as with other languages, the way to master it is to use it, and to learn by doing. Trying to do the assignment straight up means you may struggle. Work through some easier examples first.

Procedure and assessment

- The project must be completed individually and all submitted work must be your own work
- You should submit a single zip file with the following structure:
 - *<student number>_<name>.zip*
 - *model.Its* - your initial FSP model from Task 1, including the safety and liveness properties from Task 2.
 - *model_modified.Its* - your modified from Task 3, including the safety and liveness properties from Task 2.
 - *reflection.txt* - a plain text file containing your reflection from Task 4
- All source files and your reflection must contain your student number and name at the top of each file
- 1 mark per calendar day (ie, including Saturday and Sunday) will be deducted for late submissions.
- Marks will be awarded according to the rubric below.

Note: Please make sure that:

- you have acknowledged and referenced any external sources (including any GenAI) you use.
- your models and reflection are submitted in plain text.
- all files contain your name and student number.

Why backwards?

A valid question: Why are we modelling the system after implementing it? Should it not be done the other way? Well, yes and no. Many people use modelling to understand an existing code base (just look at the number of tools for reverse engineering UML models from code bases). Reverse engineering is a great way to understand problems with an existing system. It is true, however, that in many cases, it would be cheaper and easier to do the modelling first.

The other reason why the assignment is "backwards" is that trying to model a new system using a new type of notation, such as FSP, will often end in disaster. We hope that, having gone through the Java programming stage, you feel familiar with the system to be modelled and thus can concentrate on the use of FSP. The exercise should be one of applying abstraction - a skill that is of utmost importance in any engineering discipline.

▼ **View Rubric**

Select Marker

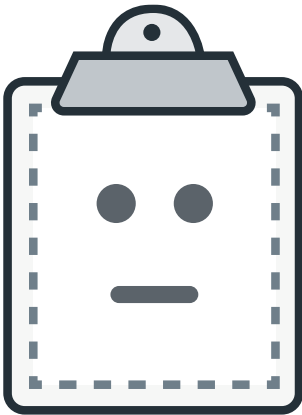
Colton Carner (TA) ▼

SWEN900041B		
Criteria	Ratings	Points

Criteria	Ratings					Points
Task 1: Correct FSP Model view longer description	Exceeds Model is faithful to specification, complete, and internally consistent. Model is clear and well-structured. 5 to >4 pts	Mastery Model is largely correct, with only very minor omissions or simplifications. Structure is generally clear. 4 to >2.5 pts	Near Model captures basic system behaviour but there are notable gaps or inaccuracies. Some 2.5 to >1 pts	Below Model is incomplete, inconsistent, or does not meaningfully represent the system. 1 to >0 pts	No Evidence Minimal or no submission. 0 pts	3.5/5 pts
Comment Great model overall! Some small notes: - Can slightly simplify Airport by removing the 'parked' variable (can get number parked planes as just MAX_RUNWAYS-rwy), which will simplify the state space of the model. Can also simplify further by not having subprocesses for airport state optionally, have a look at sample solution and note how it has fewer transition rules. - No need for separate 'max_capacity' and 'max_flights', also there's no concept of 'parked' aircraft in the model, aircraft park only at gates. - hardcoded renaming of flight actions in SYSTEM doesn't work for >3 planes. - No planes should be allowed to land while airport is closed: in your model if I 'join queue' with plane I can still land while airport is in closing state.						
Task 2: Properties view longer description	Mastery Defines appropriate safety and liveness properties that are clearly specified and meaningfully aligned with the model, demonstrating a clear understanding of concurrent behaviour. 2 to >1 pts	Near Basic properties provided, but too weak, unclear, or partially incorrect. 1 to >0 pts	No Evidence No properties provided. 0 pts			0.5/2 pts
Comment - Safety properties won't work since aren't prefixed with plane process names - Didn't compose safety properties with main process to actually test them; need to make a composed process like `  SAFE_SYSTEM = (SYSTEM    MY_SAFETY_PROPERTY).` to test them. - NO_LAND_WHEN_CLOSING doesn't need so many transitions labelled; just include the relevant ones (eg request_takeoff can be done from either `NO_LAND_CLOSED` or `NO_LAND_WHEN_CLOSING`, so has no effect). - Liveness checks again not prefixed with aircraft, and so they actually fail when tested. - safety checks for landing priority, only cloding airport when no planes, gate mutual exclusivity not included						
Task #3: Modified Model	Mastery Modified model differs by a small, plausible change, clearly violates at least one stated property, and reflects a meaningful modelling or design error. 2 to >1 pts	Near Property violation present, but change is overly simplistic or not clearly motivated. 1 to >0 pts	No Evidence No meaningful modification, or no clear property violation. 0 pts			1/2 pts
Comment Small change looks good, just allowing planes to request landing ('queue') even when airport is closed. However, your safety / liveness properties don't catch it. Your liveness properties report an error, but that's only because you haven't prefixed them properly (eg needs to be `progress FLIGHTS_LAND_MOD = {flight[0].request_land}` for instance)						

Criteria	Ratings				Points
Task #4: Reflection view longer description	Mastery Clear reference to submitted FSP model(s) across all prompts, clear explanation, precise writing, and within word limits.  3.5 to >2.5 pts	Near Addresses all prompts with reasonable clarity, some specific reference to model(s), explanations are generally sound but may lack depth or precision 2.5 to >1.5 pts	Below Addresses most prompts but explanations lack sound reasoning, depth or precision, limited reference to specific model elements. 1.5 to >0 pts	No Evidence No submission, or generic reflection with little connection to model or task. 0 pts	3/3.5 pts
Comment Reflection on the whole is well written. - Description of violation that occurs from task3 modification is incorrect (see discussion in above comment)					

File Name		Size	
	1239956_J...ang-1.zip	8.35 KB	Download (https://canvas.lms.unimelb.edu.au/files/27060677/download?download_frd=1&verifier=m3hP4TDE9cznfnpF3cHfrIE5ogN208TQjWnwhE99)



Preview Unavailable

1239956_JunesuhKang-1.zip

New Attempt